

Developer Wiki - Aegivex AI

Version:	1.0
Document Type:	Developer Wiki
Project:	OKX.AI Genesis Hackathon
Prepared For:	Development Team
Prepared By:	Aegivex AI Team

1. Overview

This document serves as the central technical guide for developers working on **Aegivex AI**. It defines the project structure, development standards, coding guidelines, Git workflow, environment setup, and best practices to ensure consistency across the entire codebase.

2. Project Structure

Frontend Structure	Backend Structure
app/ components/ hooks/ services/ lib/ styles/ public/ types/ utils/	api/ routers/ services/ models/ schemas/ database/ middleware/ prompts/ utils/ config/ tests/

3. Technology Stack

Category	Technologies
Frontend	Next.js, React, TypeScript, Tailwind CSS
Backend	FastAPI, Python
Database & ORM	PostgreSQL, SQLAlchemy, Alembic (Migrations)
AI Frameworks	OpenAI API, LangChain, LangGraph
Deployment	Vercel (Frontend), Railway / Render (Backend)
Version Control	Git

4. Local Development Setup

Required Software

- Node.js
- Python 3.11+
- PostgreSQL
- Git
- VS Code

Setup Instructions

```
# Clone Repository
git clone <repository-url>
```

```
# Install Frontend
npm install
```

```
# Install Backend
```

```
pip install -r requirements.txt
```

```
# Run Frontend  
npm run dev
```

```
# Run Backend  
uvicorn main:app --reload
```

5. Environment Variables

Scope	Variable Names
Frontend	NEXT_PUBLIC_API_URL, NEXT_PUBLIC_APP_NAME
Backend	DATABASE_URL, OPENAI_API_KEY, JWT_SECRET, JWT_EXPIRE, CORS_ORIGINS, LOG_LEVEL

Security Note: Never commit environment variables or .env files to Git repositories.

6. Coding Standards

Frontend Guidelines

- Always use **TypeScript** for strict type safety.
- Build reusable, modular components.
- Keep individual components small and focused on a single responsibility.
- Avoid duplicated code (DRY principle).

Backend Guidelines

- Follow strict **REST principles** for API design.
- Validate every incoming request payload.
- Use a service-based architecture.
- Keep core business logic completely outside of HTTP routers.

General Guidelines

- Use clear, meaningful variable and function names.
- Write clean, expressive, and easily readable code.
- Add code comments only where complex logic truly warrants explanation.

7. Git Workflow

Branch Type	Naming Convention / Example
Main Branch	main
Development Branch	develop
Feature Branch	feature/wallet-scanner
Bug Fix Branch	fix/auth-login

Commit Message Formats

feat: add wallet scanner
fix: resolve login issue
docs: update API documentation

8. Folder Responsibilities

Directory	Responsibility
app/	Application routes and page views.
components/	Reusable UI elements and layout blocks.
services/	API integration and external service

Directory	Responsibility
	communication layer.
hooks/	Custom React hooks for state and side-effects.
database/	Database models, connection strings, and ORM setup.
routers/	REST API endpoints and routing handlers.
schemas/	Data validation schemas and Pydantic models.
middleware/	Authentication checks, logging, and request processing.
prompts/	Structured AI prompt templates and system messages.
utils/	Shared utility functions and common helpers.

9. API Development Rules

- ---

Follow standardized RESTful design conventions.
- Ensure all responses return valid **JSON** format.
- Sanitize and validate every incoming request parameter.
- Return accurate HTTP status codes (e.g., 200, 201, 400, 401, 404, 500).
- Handle exceptions gracefully and present structured error payloads.

10. Database Guidelines

- ---

Use **UUIDs** as primary key identifiers for all tables.

- Create explicit database indexes on heavily queried fields.
- **Never delete production data directly**; implement soft deletes if necessary.
- Always use database migrations (Alembic) for modifying schema definitions.
- Maintain relational integrity with foreign keys and cascading rules where appropriate.

11. AI Development Guidelines

- ---

Design modular, easily customizable prompt architectures.
- Explicitly separate system-level prompts from user input content.
- Avoid returning hard-coded static AI responses.
- Force structured data outputs (e.g., JSON schemas) from LLMs.
- Log all AI model invocation failures and edge-case exceptions.

12. Security Guidelines

- ---

Never expose sensitive API keys or credentials in client bundles or public code.
- Store secrets safely in environment configurations.
- Hash user passwords using **bcrypt** before persistence.
- Protect stateful and stateless routes using **JWT authentication**.
- Sanitize all inputs to mitigate XSS and injection vulnerabilities.
- Enforce strict HTTPS protocols across production deployments.

13. Logging & Debugging

Log events for the following operations:

- API request lifecycle events
- User authentication and authorization checks
- AI responses and model metadata
- System errors and stack traces
- Performance and latency metrics

Warning: Ensure sensitive personally identifiable information (PII) is never written to log stores.

14. Testing Guidelines

Before every release, perform verification on the following core features:

- Authentication flows
- AI Chat engine
- Wallet Scanner
- Token Scanner
- Contract Scanner
- Website Scanner
- Transaction Explainer
- Dashboard interfaces
- Database operations and CRUD routines

15. Code Review Checklist

Ensure the following conditions are met prior to merging pull requests:

- ☐ Code compiles cleanly without build breaks.
- ☐ Zero runtime errors or warnings in the browser console.
- ☐ Security standards and input validation rules are strictly maintained.
- ☐ API response structures are validated against schemas.
- ☐ Database migrations run cleanly both forwards and backwards.
- ☐ User interface is responsive across multiple screen breakpoints.
- ☐ Code performance remains acceptable under load.

16. Deployment Checklist

Required tasks prior to triggering a production release:

- ☐ All production environment variables are properly set.
- ☐ Target database schema migrations have executed successfully.
- ☐ Production API endpoints pass health checks.
- ☐ Valid SSL/TLS certificates are active.
- ☐ Application error tracking/monitoring tools are active.
- ☐ Logging output channels are functioning properly.
- ☐ Latest database backup snapshot has been generated.

17. Documentation Rules

Every newly introduced feature pull request must supply documentation covering:

- High-level feature description
- API endpoint updates and schema additions
- Database changes and migration notes
- AI prompt template modifications
- Testing verification steps
- Deployment and configuration dependencies

18. Future Development

Roadmap of planned technical enhancements:

- Multi-Agent AI orchestration
- Dedicated Browser Extension interface
- Mobile Application deployment
- Enterprise Management Dashboard
- Cross-Chain protocol support
- Real-time security monitoring system

19. Expected Outcome

This Developer Wiki provides a single source of truth for the development team, ensuring consistent coding practices, maintainable architecture, secure implementation, and efficient collaboration throughout the Aegivex AI project lifecycle.